

```

Pythondef get_parity(n):
    # Parity of the lower 8 bits (even number of 1s -> 1, else 0)
    low_8 = n & 0xFF
    return 1 if bin(low_8).count('1') % 2 == 0 else 0

def get_aux_carry(op1, op2):
    # Carry from bit 3 to 4
    return 1 if ((op1 & 0xF) + (op2 & 0xF)) > 0xF else 0

def get_overflow(op1, op2, res):
    # Signed overflow: (operands same sign) AND (result diff sign)
    # 16-bit check
    sign1 = (op1 >> 15) & 1
    sign2 = (op2 >> 15) & 1
    sign_res = (res >> 15) & 1
    return 1 if (sign1 == sign2) and (sign1 != sign_res) else 0

def add_16bit(op1, op2):
    res_full = op1 + op2
    res_16 = res_full & 0xFFFF

    cf = 1 if res_full > 0xFFFF else 0
    zf = 1 if res_16 == 0 else 0
    sf = (res_16 >> 15) & 1
    of = get_overflow(op1, op2, res_16)
    pf = get_parity(res_16)
    af = get_aux_carry(op1, op2)

    return res_16, cf, zf, sf, of, pf, af

# Q10: ABCDh + 9876H
q10_res = add_16bit(0xABCD, 0x9876)
print(f"Q10: Result={hex(q10_res[0])}, CF={q10_res[1]}, ZF={q10_res[2]},
SF={q10_res[3]}, OF={q10_res[4]}, PF={q10_res[5]}, AF={q10_res[6]}")

# Q11: 2FXYh + FCDh. PF=1. Min X, Max Y.
print("Q11 Analysis:")
found_q11 = []
for x in range(16):
    for y in range(16):
        op1 = (0x2F00) + (x << 4) + y
        op2 = 0xFCD
        _, _, _, _, _, pf, _ = add_16bit(op1, op2)
        if pf == 1:
            found_q11.append((x, y))

# Sort to find min X, then for that X, find max Y? The question asks "Min
value for X and Max value for Y".
# Usually means minimize X first, then maximize Y. Or independently? "Min
X AND Max Y".
# Let's look at all valid pairs.
min_x = min(f[0] for f in found_q11)
valid_for_min_x = [f for f in found_q11 if f[0] == min_x]
max_y_for_min_x = max(f[1] for f in valid_for_min_x)
print(f"Min X={hex(min_x)}, Max Y (given Min X)={hex(max_y_for_min_x)}")

```

```

# Calculate flags for this case
op1_q11 = (0x2F00) + (min_x << 4) + max_y_for_min_x
q11_res = add_16bit(op1_q11, 0xFCDF)
print(f"  Flags: OF={q11_res[4]}, AF={q11_res[6]}, SF={q11_res[3]}")

# Q12: DCBAh + 4567h
q12_res = add_16bit(0xDCBA, 0x4567)
print(f"Q12: Result={hex(q12_res[0])}, CF={q12_res[1]}, ZF={q12_res[2]},
SF={q12_res[3]}, OF={q12_res[4]}, PF={q12_res[5]}, AF={q12_res[6]}")

# Q13: C5YX + 5B6A. PF=1. Min X and Y.
print("Q13 Analysis:")
found_q13 = []
for x in range(16):
    for y in range(16):
        # C5YX -> C500 + Y*16 + X? Notation usually XY means HighNibble
        LowNibble.
        # "C5YX" implies Y is upper nibble of lower byte, X is lower
        nibble?
        # Usually it's HighByte LowByte. Digits are 4 bits.
        # "C5YX" -> C=12, 5=5, Y, X. So Value = C500 + (Y<<4) + X.
        op1 = 0xC500 + (y << 4) + x
        op2 = 0x5B6A
        _', _', _', _', _', pf, _ = add_16bit(op1, op2)
        if pf == 1:
            found_q13.append((x, y))
# "Min values of X and Y". Usually implies minimizing the number formed by
# YX or finding min X and min Y individually if possible.
# Let's assume we want to minimize the value YX (i.e. minimize Y then X).
# Or "Min X and Min Y" implies finding a pair where both are minimal.
# Let's find the pair that minimizes (Y<<4 + X).
found_q13.sort(key=lambda p: (p[1]<<4) + p[0])
best_q13 = found_q13[0]
print(f"  Min Y={hex(best_q13[1])}, Min X={hex(best_q13[0])}")
q13_res = add_16bit(0xC500 + (best_q13[1]<<4) + best_q13[0], 0x5B6A)
print(f"  Flags: SF={q13_res[3]}, OF={q13_res[4]}, AF={q13_res[6]},
ZF={q13_res[2]}")

# Q14: AFCEh + FFXYh. PF=1, AF=1. FFXY max.
print("Q14 Analysis:")
found_q14 = []
for x in range(16):
    for y in range(16):
        # FFXY -> FF00 + (X<<4) + Y
        op2 = 0xFF00 + (x << 4) + y
        op1 = 0xAFCE
        _', _', _', _', _', pf, af = add_16bit(op1, op2)
        if pf == 1 and af == 1:
            found_q14.append((x, y))
# Maximize FFXY -> Maximize (X<<4) + Y
found_q14.sort(key=lambda p: (p[0]<<4) + p[1], reverse=True)
best_q14 = found_q14[0]

```

```

print(f"  Max X={hex(best_q14[0])}, Max Y={hex(best_q14[1])}")
q14_res = add_16bit(0xAFCF, 0xFF00 + (best_q14[0]<<4) + best_q14[1])
print(f"  Flags: OF={q14_res[4]}, ZF={q14_res[2]}, SF={q14_res[3]},
CF={q14_res[1]}")

# Q15: 96C9 + 99XY. AF=1. Min X and Y.
print("Q15 Analysis:")
found_q15 = []
for x in range(16):
    for y in range(16):
        # 99XY -> 9900 + (X<<4) + Y
        op1 = 0x96C9
        op2 = 0x9900 + (x << 4) + y
        _, _, _, _, _, af = add_16bit(op1, op2)
        if af == 1:
            found_q15.append((x, y))
# Min X and Y -> Minimize (X<<4) + Y
found_q15.sort(key=lambda p: (p[0]<<4) + p[1])
best_q15 = found_q15[0]
print(f"  Min X={hex(best_q15[0])}, Min Y={hex(best_q15[1])}")
q15_res = add_16bit(0x96C9, 0x9900 + (best_q15[0]<<4) + best_q15[1])
print(f"  Flags: PF={q15_res[5]}, SF={q15_res[3]}, OF={q15_res[4]},
CF={q15_res[1]}")

# Memory Calculations

# Q17
# CS=2000h, DS=3000h, SS=2000h, BX=1000h, BP=2000h, IP=3456h, SI=3000h
# MOV DX, [BP+SI+2000h]
# a) Fetch location: Always CS:IP
cs_17 = 0x2000
ip_17 = 0x3456
fetch_addr_17 = (cs_17 * 16) + ip_17
print(f"Q17 Fetch PA: {hex(fetch_addr_17)}")

# b) Data stored in DX
# EA = BP + SI + 2000h. BP uses SS by default.
bp_17 = 0x2000
si_17 = 0x3000
disp_17 = 0x2000
ea_17 = (bp_17 + si_17 + disp_17) & 0xFFFF # Offset wrap
ss_17 = 0x2000
pa_17 = (ss_17 * 16) + ea_17
print(f"Q17 Data PA: {hex(pa_17)}")
# Table data for Q17:
# 07000h: 12h, 07001h: 34h -> Word 3412h? Or 1234h? Little Endian.
# 27000h: 56h, 27001h: 78h
# 37000h: 10h, 37001h: 20h
# Check which one PA_17 matches.

# Q18
# DS=1000h, SS=2000h, CS=3000h, BP=0500h, SI=0100h
# MOV BX, [BP+SI]
```

```

# BP implies SS.
ss_18 = 0x2000
bp_18 = 0x0500
si_18 = 0x0100
ea_18 = (bp_18 + si_18) & 0xFFFF
pa_18 = (ss_18 * 16) + ea_18
print(f"Q18 Data PA: {hex(pa_18)}")
# Table Q18:
# 10600h:12, 10601h:34
# 20600h:56, 20601h:78
# 30600h:10, 30601h:20

# Q19
# DS=1000h, CS=3000h, SS=8A40h, SI=2000h, CX=43AEh, DI=030Fh
# JMP [SI+1000h]
# Indirect Memory Near Jump? Or Far Jump? "JMP [SI+1000h]" usually fetches
target IP from memory.
# Segment for data access [SI+disp] is DS (SI default is DS).
ds_19 = 0x1000
si_19 = 0x2000
disp_19 = 0x1000
ea_19 = (si_19 + disp_19) & 0xFFFF
pa_19_data = (ds_19 * 16) + ea_19
print(f"Q19 Pointer Location PA: {hex(pa_19_data)}")
# Table Q19:
# 14000h: 56h, 14001h: 78h ...
# Fetched word will be new IP (if near jump).
# The question asks "Deduce the physical address of the instruction that
8086 will jump to."
# So we need (CS_current * 16) + New_IP? Or does it fetch CS too?
# Standard "JMP [mem]" is Near Indirect (fetches IP only) unless specified
"JMP FAR [mem]".
# Assuming Near. Target = CS_old : New_IP.

# Q20
# DS=1000h, CS=3000h, BX=2000h
# JMP [BX]
# Data segment for BX is DS.
ds_20 = 0x1000
bx_20 = 0x2000
ea_20 = bx_20
pa_20_ptr = (ds_20 * 16) + ea_20
print(f"Q20 Pointer Location PA: {hex(pa_20_ptr)}")
# Table Q20:
# 12000h: 10h, 12001h: 20h -> Word 2010h

```

Code output

```

Q10: Result=0x4443, CF=1, ZF=0, SF=0, OF=1, PF=0, AF=1
Q11 Analysis:
    Min X=0x0, Max Y (given Min X)=0xf
    Flags: OF=0, AF=1, SF=0
Q12: Result=0x2221, CF=1, ZF=0, SF=0, OF=0, PF=1, AF=1
Q13 Analysis:

```

```

Min Y=0x0, Min X=0x0
Flags: SF=0, OF=0, AF=0, ZF=0
Q14 Analysis:
Max X=0xf, Max Y=0xd
Flags: OF=0, ZF=0, SF=1, CF=1
Q15 Analysis:
Min X=0x0, Min Y=0x7
Flags: PF=0, SF=0, OF=1, CF=1
Q17 Fetch PA: 0x23456
Q17 Data PA: 0x27000
Q18 Data PA: 0x20600
Q19 Pointer Location PA: 0x13000
Q20 Pointer Location PA: 0x12000

```

Here are the comprehensive answers to the questions in the practice sheet, verified for accuracy.

Intro to Microprocessors

1. Differences between Microprocessors and Microcontrollers:

- **Microprocessor (μP):** A general-purpose CPU (e.g., Intel Core i7). It requires external components like RAM, ROM, and I/O ports to function. It is designed for high processing power and flexibility.
- **Microcontroller (μC):** A dedicated computer on a chip (e.g., Arduino/ATmega328). It integrates the CPU, RAM, ROM, and I/O ports on a single chip. It is designed for specific control tasks, low power consumption, and cost-efficiency.

2. Major components of a CPU:

- **ALU (Arithmetic Logic Unit):** Performs mathematical (add, subtract) and logical (AND, OR) operations.
- **CU (Control Unit):** Decodes instructions and generates control signals to coordinate data flow between CPU components and memory/I/O.
- **Registers:** Small, high-speed storage locations (e.g., Accumulator, Program Counter) used to hold temporary data and addresses during execution.

3. Choice of Device:

- a. **Keyboard:** Microcontroller (Simple input scanning).
- b. **Mouse:** Microcontroller (Simple sensor data processing).
- c. **Headphone:** None (Passive) or Microcontroller (if Wireless/Noise Cancelling).
- d. **Computer:** Microprocessor (High performance required).
- e. **TV Remote:** Microcontroller (Low power, specific task).
- f. **Smart TV:** Microprocessor (Complex OS, graphics, internet).
- g. **Regular fridge:** Microcontroller (Simple temperature control).
- h. **Mobile phone:** Microprocessor (High performance for OS/Apps).

4. Smart Home System:

- **Choice: Microcontroller.**
- **Justification:** Smart home nodes (like sensors or switches) perform specific, repetitive tasks (reading sensors, toggling lights). They require low power consumption to run on batteries and are cost-sensitive. A full microprocessor would be overkill and power-inefficient.

5. 8-bit vs 4-bit Performance:

- **Explanation:** Bit-width (8-bit vs 4-bit) refers to the amount of data processed in one cycle. While an 8-bit processor handles more data at once, speed also depends on **Clock Speed (Frequency)**, **Architecture (RISC vs CISC)**, **Pipelining**, and **Instruction Cycle count**. An 8-bit processor running at a very low clock speed could be slower than a highly optimized 4-bit processor running at a high speed for specific tasks.

6. Washing Machine Embedded System:

- **Why Microcontroller:** A washing machine needs to interface directly with sensors (water level, temp) and actuators (motors, valves). A microcontroller has built-in **ADC (Analog-to-Digital Converters)**, **GPIO pins**, and **Timers** necessary for this control. It is also cheaper and more compact.

7. High-Performance Computing (RISC vs CISC):

- **Recommendation: RISC (Reduced Instruction Set Computer).**
- **Why:** RISC architectures use simple, single-cycle instructions that allow for effective **pipelining** and higher clock speeds. This generally leads to higher throughput and performance for computational tasks compared to CISC, which focuses on code density and complex instructions.

8. Consultant Choice (Graphics, Games, Power Balance):

- **Choice: Microprocessor** (specifically a mobile/embedded microprocessor or Application Processor).
- **Reason:** High processing power for graphics and game logic exceeds the capabilities of standard microcontrollers. While power consumption is a concern, modern microprocessors have power-saving modes, whereas microcontrollers simply lack the raw performance required for high-end graphics.

9. Assembly Program:

Code snippet

```
MOV AX, 7    ; Store 7 in a register (AX)
ADD AX, 3    ; Add 3 to it (AX = 10)
SUB AX, 5    ; Subtract 5 (AX = 5)
```

Overview of Microcomputer

1. Block Diagram: (Conceptual)

- **CPU** <==> **System Bus** (Address, Data, Control) <==> **Memory** (RAM/ROM) & **I/O**

2. RAM vs ROM:

- **RAM (Random Access Memory):** Read/Write memory. **Volatile** (loses data when power is lost). Used for temporary execution data.
- **ROM (Read Only Memory):** Read-only (mostly). **Non-Volatile** (retains data without power). Used for startup code (BIOS).

3. System Buses:

- **Address Bus:** Carries memory location addresses (Unidirectional: CPU $\rightarrow$ Memory).
- **Data Bus:** Carries actual data (Bidirectional).
- **Control Bus:** Carries command signals like Read/Write (Unidirectional/Bidirectional depending on line).

4. Calculation Handler: ALU (Arithmetic Logic Unit).

5. Address Bus Role: Identifies the specific memory location or I/O device the CPU wants to access.

6. Temporary Data Component: Registers (specifically General Purpose Registers).

7. CPU Functions:

- **Fetch:** Retrieve instruction from memory.
- **Decode:** Interpret what the instruction does.
- **Execute:** Perform the operation (math, data move, etc.).

8. Unidirectional Address Bus: The CPU is the *master* that decides *where* to read/write. Memory/I/O devices are *slaves* that simply respond. They do not tell the CPU where to look; hence, addresses flow only *out* from the CPU.

9. Startup Instructions: ROM. Because startup instructions (BIOS/Bootloader) must be present immediately upon power-up and must not be lost when power is turned off.

10. Program Counter (PC): It holds the address of the **next** instruction to be fetched. After a fetch, it automatically increments to point to the subsequent instruction.

11. MAR vs MDR:

- **MAR (Memory Address Register):** Holds the *address* of the location to be accessed.
- **MDR (Memory Data Register):** Holds the *data* fetched from or to be written to that address.

12. 16-bit Address Bus Capacity: $2^{16} = 65,536$ locations (64 KB).

13. Delay Cause: The **Decode** or **Execute** phase. Complex instructions (CISC) take more clock cycles to decode and execute than simple ones.

14. Increasing Data Bus Size: Yes. A wider data bus (e.g., 16-bit vs 8-bit) can transfer more data in a single cycle, reducing the number of fetch cycles needed for large data transfers, thus improving performance.

15. Control Bus Role: It synchronizes the timing. It sends "Read" or "Write" signals so the external device knows *when* to place data on the bus or grab data from it, preventing data collisions or missed data.

8086 Memory Address Space Partition

1. 4MB Memory Address Bus: $4 \text{ MB} = 2^{22} \text{ bytes}$. Bus size = **22 bits**.

2. 21-bit Address Bus Memory: $2^{21} = 2 \text{ MB}$.

3. Logical vs Physical Address:

- **Logical:** `Segment:Offset` (e.g., `2000h:1000h`). Used in programming.
- **Physical:** 20-bit actual address (e.g., `21000h`). Calculated as `Segment * 10h + Offset`.

4. Segment Registers: Registers (CS, DS, SS, ES) that point to the base of 64KB memory blocks. They allow the 16-bit CPU to access 1MB of memory by dividing it into segments.

5. Overlapping vs Non-overlapping:

- **Non-overlapping:** Segments start every 64KB (e.g., Seg 0 @ `00000h`, Seg 1 @ `10000h`).
- **Overlapping:** Segments start within 64KB of each other (e.g., Seg A @ `10000h`, Seg B @ `10010h`).

6. Advantages of Segmentation: Allows 1MB addressing with 16-bit registers; Relocatable code; Separation of Code, Data, and Stack.

7. Segment Registers Purpose: Define starting addresses for different program sections. Different from General Purpose registers because they cannot be used for arithmetic (ADD/SUB) and are strictly for address calculation.

8. Physical Address Calc: Segment `A4FBh`, Offset `4872h`.

$$PA = (A4FBh \times 10h) + 4872h = A4FB0h + 4872h = \text{A9822H}$$

9. Overlapping Check: 20000H and 20100H.

- **Yes, Overlapping.** The first segment ranges from 20000H to 2FFFFH. The second starts at 20100H, which is inside the first segment's range.

10. Physical Address: Seg 1111H, Off 1332H.

$$PA = 11110h + 1332h = \text{12442H}$$

11. Segment Number: PA 33330H, Off 0020H.

$$Seg = (33330h - 0020h) / 10h = 33310h / 10h = \text{3331H}$$

12. Logical Address Conversion:

- **A9822H:** A982h:0002h (Many answers possible).
- **38A41H:** 38A4h:0001h.

13. Segments in 1MB: $1 \text{ MB} / 64 \text{ KB} = 16$ non-overlapping segments.

14. CS:IP Physical: CS 4321H, IP 1234H.

$$PA = 43210h + 1234h = \text{44444H}$$

15. 5th Largest/Smallest for 32556h:

- 5th Largest Segment: Maximize Base $\rightarrow$ Minimize Offset.

Offsets: 0, 1, 2, 3, 4, 5, 6.

$$PA - 6 = 32550h \rightarrow Seg = \text{3255h} \text{ (Logical: 3255h:0006h)}$$

- 5th Smallest Segment: Minimize Base $\rightarrow$ Maximize Offset.

Max Offset FFFFh. $32556h - FFFFh = 22557h$.

Smallest valid Base ending in 0: $22560h \rightarrow Seg \text{ 2256h}$.

Sequence: 2256h, 2257h, 2258h, 2259h, 225Ah.

$$Seg = \text{225Ah} \text{ (Logical: 225Ah:FFB6h)}$$

16. Two Logical Addresses for 78F97h:

1. 78F9h:0007h
2. 7000h:8F97h

17. Memory Calculation (Table 1):

- Context: \$CS=2000h, SS=2000h, BP=2000h, SI=3000h, IP=3456h\$.
- Instruction: `MOV DX, [BP+SI+2000h]`
- a. Fetch Location: Depends on CS and IP.

$$\$PA = (CS \times 10h) + IP = 20000h + 3456h = \texttt{\textbf{23456h}}\$$$

- b. Data in DX:

$$\text{Effective Addr} = \$BP + SI + 2000h = 2000h + 3000h + 2000h = 7000h\$.$$

Segment: BP defaults to SS (\$2000h\$).

$$\$PA = (2000h \times 10h) + 7000h = 20000h + 7000h = \texttt{\textbf{27000h}}\$$$

From Table: Address 27000h holds 56h, 27001h holds 78h.

Since 8086 is Little Endian, DX = HighByte LowByte = 78h 56h.

$$DX = 7856h.$$

18. Memory Calculation (Table 2):

- Context: \$SS=2000h, BP=0500h, SI=0100h\$.
- Instruction: `MOV BX, [BP+SI]`
- Effective Addr = \$0500h + 0100h = 0600h\$.
- Segment: BP defaults to SS.

$$\$PA = (2000h \times 10h) + 0600h = \texttt{\textbf{20600h}}\$$$

- From Table: 20600h is 56h, 20601h is 78h.

$$BX = 7856h.$$

19. Jump Calculation:

- Context: \$DS=1000h, SI=2000h\$.
- Instruction: `JMP [SI+1000h]` (Near Indirect Jump).
- Pointer Address: \$EA = SI + 1000h = 3000h\$. Seg = DS (\$1000h\$).

$$\$PA_{\text{ptr}} = 10000h + 3000h = \texttt{\textbf{13000h}}\$$$

- From Table: 13000h is 12h, 13001h is 34h.
- New IP fetched = 3412h.
- Target Address = \$CS:NewIP = 3000h:3412h\$ (Assuming CS stays 3000h).

$$\$PA_{\{jump\}} = 30000h + 3412h = \texttt{\textbf{33412h}}\$$$

20. Jump Calculation:

- Context: \$DS=1000h, BX=2000h, CS=3000h\$.
- Instruction: `JMP [BX]`
- Pointer Address: \$EA = 2000h\$. Seg = DS.

$$\$PA_{\{ptr\}} = 10000h + 2000h = \texttt{\textbf{12000h}}\$$$

- From Table: 12000h is 10h, 12001h is 20h.
- New IP = 2010h.
- Target Address = \$CS:NewIP\$.

$$\$PA_{\{jump\}} = 30000h + 2010h = \texttt{\textbf{32010h}}\$$$

Microprocessor Internal Architecture

1-8. Concepts:

- **Functional Units:** BIU (Bus Interface Unit) handles transfers; EU (Execution Unit) executes instructions.
- **Instruction Queue:** Stores pre-fetched instructions to speed up processing (Pipelining). Fails/Flushed during a **JUMP** instruction.
- **Prefetch:** BIU fetches if queue has room (2 bytes usually). If 5 bytes filled in 6-byte queue (1 empty), prefetch typically waits for 2 bytes space in 8086.
- **Fetching Unit: BIU.**
- **Queue Size: 6 Bytes** (for 8086).

9. Register Categories:

- a. Segment Register: **BIU**
- b. Instruction Pointer: **BIU** (controls fetching)
- c. Index Register (SI, DI): **EU**
- d. General Purpose (AX..): **EU**
- e. Instruction Queue: **BIU**
- f. Flags: **EU**

Flags:

1. Size: 16 bits (9 active flags: 6 status, 3 control).

2. Status vs Control: Status reflects result of operations (Zero, Carry); Control directs CPU behavior (Direction, Interrupt).

3. ZF (Zero Flag): Set to 1 if result is zero. Useful for loops/comparisons.

4. DF (Direction Flag): Controls string processing direction (0=Increment/Forward, 1=Decrement/Backward).

5. TF (Trap Flag): Enables single-step execution for debugging.

6. Reverse String: Set DF to 1.

7. IF=0 Interrupt: No. CPU ignores maskable interrupts if IF is cleared.

8. Nibble Carry: AF (Auxiliary Carry Flag).

9. Single Step: TF (Trap Flag).

10. Flag Calculation: AX=ABCDh, BX=9876h. ADD AX, BX.

- Result: $ABCD + 9876 = 14443h$. AX = 4443h.
- CF=1 (Carry out of MSB)
- ZF=0 (Result not zero)
- SF=0 (MSB is 0)
- PF=0 (Lower byte 43h 01000011 has 3 ones $\rightarrow$ Odd)
- OF=1 (Neg + Neg = Pos result. Overflow occurred).

11. Min X, Max Y for PF=1:

- $2FXYh + FCDFh$.
- Parity depends on lower byte: $XY + DF$.
- To get PF=1, we need even number of 1s in result byte.
- **Min X = 0.**
- With X=0, we need Max Y.

Sum: $0Y + DF$.

If Y=F: $0F + DF = EE$ (1110 1110, 6 ones $\rightarrow$ Even).

Values: X=0, Y=F.

- Flags: Result $2F0F + FCDF = 12BEF$.

OF=0, AF=1, SF=0.

12. ADD DCBAh, 4567h:

- Sum: 12221h. $AX=2221h$.
- **CF=1. ZF=0. SF=0.**
- **PF=1** (Lower 21h is 00100001 $\rightarrow$ 2 ones $\rightarrow$ Even).
- **OF=0** (Neg + Pos = Pos $\rightarrow$ No overflow possible).
- **AF=1** ($A+7=17 \rightarrow$ Carry).

13. ADD C5YX, 5B6A:

- a. Min X, Y for PF=1:

Minimize pair (Y, X).

If X=0, Y=0: C500 + 5B6A ends in 6A (01101010 $\rightarrow$ 4 ones $\rightarrow$ Even).

Values: X=0, Y=0.

- b. Flags: Result 1206Ah.

SF=0 (Pos), OF=0 (Neg + Pos), AF=0 (0+A no carry), ZF=0.

14. ADD AFCh, FFXYh:

- a. PF=1, AF=1, Max FFXY:

We need Max X, Max Y.

Test X=F, Y=D (Operand FFFD).

CF + FD = 1CC (Lower CC is even parity; Aux carry F+D=1C is yes).

Values: X=F, Y=D.

- b. Flags: AFCh + FFFD = 1AFCC.

OF=0 (Neg+Neg=Neg AFCC), ZF=0, SF=1, CF=1.

15. ADD 96C9, 99XY:

- a. AF=1, Min X, Y:

Nibble sum 9+Y must carry (>15).

Min Y for 9+Y > 15 is 7 (9+7=16).

Min X is 0.

Values: X=0, Y=7.

- b. Flags: 96C9 + 9907 = 12FD0h.

PF=0 (D0 is 11010000 $\rightarrow$ 3 ones), SF=0, OF=1 (Neg+Neg=Pos), CF=1.

Addressing Modes and Machine Codes

1. Addressing Modes:

- a. [BX]: **Register Indirect.**
- b. [BX+SI]: **Based Indexed.**
- c. RET: **Implicit/Stack?** (Control Transfer).
- d. [BP]: **Register Indirect (Based).**
- e. IN: **I/O Direct.**
- f. OUT: **I/O Direct.**
- g. [BX+SI+Disp]: **Based Indexed with Displacement.**
- h. [2000H]: **Direct.**
- i. MOV Immed: **Immediate.**
- j. [BP]: **Register Indirect.**
- k. HLT: **Implied.**
- l. MOV [BX], [...]: **Illegal** (Memory to Memory).

3. Assembly to Machine Code (Hex):

- a. MOV AX, [1234H]: **A1 34 12**
- b. MOV 5678H[DI], CL: **88 8D 78 56**
- c. MOV [BX+SI+5312h], AX: **89 80 12 53**
- d. MOV BX, [BP+SI+23FFh]: **8B BA FF 23**
- e. **Illegal Instruction**
- f. MOV AX, [BX]: **8B 07**
- g. MOV 1234h[SI], AX: **89 84 34 12** (Src AX to Dest Mem)
- h. MOV DI, [BP+42h]: **8B 7E 42**
- i. MOV [C5A4h], CX: **89 0E A4 C5**

2. Machine Code Decode:

- a. **8B 54 75**: MOV DX, [SI+75h]
- b. **88 16 00 80**: MOV [8000h], DL
- c. **8B D0**: MOV DX, AX
- d. **8B DA**: MOV BX, DX
- e. **88 16 FF 96**: MOV [96FFh], DL